МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №4

з дисципліни "Функціональне програмування"

НА ТЕМУ "РЕКУРСІЯ ТА МЕМОІЗАЦІЯ"

Варіант 12. Найдовша спільна підпослідовність (LCS)

Виконав:
студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів — 2026

Мета: зрозуміти структуру рекурсивних обчислень (базовий випадок і крок), обмеження стеку викликів у Python і причини відсутності tail-call optimization (TCO), навчитися мемоізувати підзадачі за допомогою functools.lru_cache; реалізувати рекурсивний пошук найдовшої спільної підпослідовності (LCS) для двох рядків з мемоізацією станів (варіант 12) та її ітеративну альтернативу, а також базові алгоритми: quicksort, обходи бінарного дерева та мемоізований Фібоначчі.

Теоретичні відомості

Рекурсивний алгоритм зводить розв'язання вихідної задачі до вирішення однієї або кількох ідентичних підзадач меншого розміру. Ключовими складовими рекурсії є *базовий випадок* (умова виходу без подальших викликів) та *рекурсивний крок* (виклики функції над зменшеними даними). Кожен виклик виділяє стековий кадр. У CPython ліміт глибини рекурсії становить за замовчуванням близько 1000 кадрів (`sys.getrecursionlimit()`), а його перевищення призводить до помилки `RecursionError`. Оптимізація хвостової рекурсії (TCO) у CPython свідомо не реалізована задля збереження інформативності трасування стека помилок, тому для глибоких структур обов'язковим є використання ітеративних схем (стек/черга в купі).

Мемоізація зберігає вже обчислені значення чистих функцій у хеш-таблиці. Для задач із перекривними підзадачами (як-от обчислення LCS чи чисел Фібоначчі) декоратор `@lru_cache` оптимізує час виконання з експоненційного $O(2^{n+m})$ до поліноміального $O(n \cdot m)$. Функції `cache_info()` та `cache_clear()` забезпечують спостережуваність і керування кешем під час тестування.

Хід роботи

1. Реалізація варіанта 12 (lcs.py): реалізовано рекурсивне обчислення довжини LCS для двох рядків. У рекурсивному підході базовим випадком є вихід індексу за межі рядка (повертає 0). Рекурсивний крок: при збігу символів — `1 + rec(i+1, j+1)`; якщо символи відрізняються — `max(rec(i+1, j), rec(i, j+1))`. Застосовано `@lru_cache` над парою станів `(i, j)` та реалізовано наївну версію для порівняння:

```
from __future__ import annotations
from functools import lru_cache

def lcs_naive(s1: str, s2: str) -> int:
    """Наївна рекурсивна реалізація LCS без кешування: O(2^(n+m))."""
    def _rec(i: int, j: int) -> int:
        if i == len(s1) or j == len(s2):
            return 0
        if s1[i] == s2[j]:
            return 1 + _rec(i + 1, j + 1)
        return max(_rec(i + 1, j), _rec(i, j + 1))
    return _rec(0, 0)

def lcs_memo(s1: str, s2: str) -> int:
    """Рекурсивна реалізація LCS із мемоізацією станів (i, j): O(n * m)."""
    @lru_cache(maxsize=100_000)
    def _rec(i: int, j: int) -> int:
        if i == len(s1) or j == len(s2):
            return 0
        if s1[i] == s2[j]:
```

```
            return 1 + _rec(i + 1, j + 1)
        return max(_rec(i + 1, j), _rec(i, j + 1))

    res = _rec(0, 0)
    lcs_memo._last_cache_info = _rec.cache_info
    lcs_memo._last_cache_clear = _rec.cache_clear
    return res
```

2. Ітеративна альтернатива (lcs.py): реалізовано динамічне програмування знизу-вгору (bottom-up tabulation) з оптимізацією просторової складності до двох рядків DP-масиву *O(min(n, m))*. Вона не навантажує стек викликів CPython і безпечна для рядків довільної довжини:

```
def lcs_iterative(s1: str, s2: str) -> int:
    """Ітеративна версія з оптимізацією пам'яті: O(n * m) часу, O(min(n, m))
пам'яті."""
    if len(s1) < len(s2):
        s1, s2 = s2, s1
    prev = [0] * (len(s2) + 1)
    for ch1 in s1:
        curr = [0] * (len(s2) + 1)
        for j, ch2 in enumerate(s2, 1):
            if ch1 == ch2:
                curr[j] = prev[j - 1] + 1
            else:
                curr[j] = max(prev[j], curr[j - 1])
        prev = curr
    return prev[len(s2)]
```

3. Базові алгоритми лабораторної (algorithms.py): реалізовано чистий рекурсивний quicksort із випадковим вибором опорного елемента (pivot), що запобігає деградації до $O(n^2)$ на впорядкованих послідовностях, структуру вузла бінарного дерева `Node` та рекурсивні генератори його обходу (preorder, inorder, postorder):

```
from dataclasses import dataclass
from random import choice
from typing import Iterator

def quicksort(xs: list[int]) -> list[int]:
    if len(xs) <= 1:
        return xs[:]
    p = choice(xs)
    left = [x for x in xs if x < p]
    mid = [x for x in xs if x == p]
    right = [x for x in xs if x > p]
    return quicksort(left) + mid + quicksort(right)

@dataclass(frozen=True, slots=True)
class Node:
    key: int
    left: Node | None = None
    right: Node | None = None

def preorder(t: Node | None) -> Iterator[int]:
    if t is None: return
    yield t.key
    yield from preorder(t.left)
    yield from preorder(t.right)

def inorder(t: Node | None) -> Iterator[int]:
    if t is None: return
    yield from inorder(t.left)
```

```
    yield t.key
    yield from inorder(t.right)

def postorder(t: Node | None) -> Iterator[int]:
    if t is None: return
    yield from postorder(t.left)
    yield from postorder(t.right)
    yield t.key
```

4. Ітеративні обходи дерева та мемоізований Фібоначчі: реалізовано обходи `bfs_level_order` (через deque) і `dfs_preorder_iter` (через явний стек), а також функції обчислення чисел Фібоначчі: `fib` з декоратором `@lru_cache`, наївний рекурсивний варіант `fib_naive` та ітеративний цикл `fib_iter` зі складністю по пам'яті *O(1)*:

```
from collections import deque

def bfs_level_order(t: Node | None) -> list[int]:
    if not t: return []
    q, out = deque([t]), []
    while q:
        n = q.popleft()
        out.append(n.key)
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)
    return out

def dfs_preorder_iter(t: Node | None) -> list[int]:
    if not t: return []
    stack, out = [t], []
    while stack:
        n = stack.pop()
        out.append(n.key)
        if n.right: stack.append(n.right)
        if n.left: stack.append(n.left)
    return out

@lru_cache(maxsize=100_000)
def fib(n: int) -> int:
    if n < 0: raise ValueError("n must be non-negative")
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def fib_iter(n: int) -> int:
    if n < 0: raise ValueError("n must be non-negative")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

5. Модульне тестування (pytest): у директорії tests/ реалізовано набір тестів: параметризовані перевірки граничних випадків (порожні рядки, повні неспівпадіння, ідентичні рядки), перевірка збігу результатів рекурсивного кешованого та ітеративного алгоритмів, робота API кешу (hits, misses, currsize після cache_clear), а також тести для quicksort, обходів дерева та чисел Фібоначчі.

```
def test_lcs_cache_api() -> None:
    s1, s2 = "BANANA", "ATANA"
    assert lcs_memo(s1, s2) == 4
    info = lcs_memo._last_cache_info()
    assert info.hits > 0
    assert info.misses > 0
```

```
    lcs_memo._last_cache_clear()
    assert lcs_memo._last_cache_info().currsize == 0

def test_parity_memo_and_iterative() -> None:
    s1 = "DYNAMIC_PROGRAMMING_AND_RECURSION"
    s2 = "MEMOIZATION_WITH_LRU_CACHE_TEST"
    assert lcs_memo(s1, s2) == lcs_iterative(s1, s2)
```

6. Демонстрація роботи програми та вимірювання швидкодії (main.py): виконано замір часу обчислення LCS та чисел Фібоначчі з мемоізацією та без неї на ідентичних вхідних даних:

```
C:\Users\markian\Desktop\labs\FunctionalProgramming\lab04> python main.py
=== Демонстрація Варіанта 12 (LCS) ===
LCS("AGGTAB", "GXTXAYB") -> 4
lcs_memo("AGGTAB1234", "GXTXAYB123"): 0.048 ms
lcs_naive("AGGTAB1234", "GXTXAYB123"): 12.842 ms
Прискорення за рахунок мемоізації: 267.5x
cache_info: CacheInfo(hits=68, misses=83, maxsize=100000, currsize=83)

fib(30) cached: 0.021 ms
fib(30) naive: 89.410 ms
fib(90) result: 2880067194370816120 | fib_iter(90): 2880067194370816120
Quicksort test: [1, 1, 2, 3, 4, 5, 6, 9]
Tree Inorder: [1, 2, 3, 4, 6, 7] | BFS: [4, 2, 6, 1, 3, 7]
CPython recursion limit: 1000
```

7. Результати виконання перевірок якості коду (pytest, mypy, ruff, black):

```
C:\Users\markian\Desktop\labs\FunctionalProgramming\lab04> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.3, pytest-8.3.2, pluggy-1.5.0
collected 20 items

tests\test_fib.py::test_fib_small PASSED                                 [  5%]
tests\test_fib.py::test_fib_iter_matches PASSED                          [ 10%]
tests\test_fib.py::test_cache_clear_and_info PASSED                      [ 15%]
tests\test_fib.py::test_negative_rejected PASSED                         [ 20%]
tests\test_lcs.py::test_empty_cases PASSED                               [ 25%]
tests\test_lcs.py::test_matching_and_disjoint PASSED                     [ 30%]
tests\test_lcs.py::test_standard_subsequences PASSED                     [ 35%]
tests\test_lcs.py::test_lcs_cache_api PASSED                             [ 40%]
tests\test_lcs.py::test_parity_memo_and_iterative PASSED                 [ 45%]
tests\test_lcs.py::test_input_not_mutated PASSED                         [ 50%]
tests\test_quicksort.py::test_quicksort_basic PASSED                     [ 55%]
tests\test_quicksort.py::test_quicksort_random PASSED                    [ 60%]
tests\test_quicksort.py::test_quicksort_does_not_mutate PASSED           [ 65%]
tests\test_quicksort.py::test_matches_sorted PASSED                      [ 70%]
tests\test_tree.py::test_traversals PASSED                               [ 75%]
tests\test_tree.py::test_bfs_and_dfs_iter PASSED                         [ 80%]
tests\test_tree.py::test_empty_tree PASSED                               [ 85%]
tests\test_tree.py::test_deep_tree_iterative PASSED                      [ 90%]
tests\test_benchmark.py::test_benchmark_lcs PASSED                       [ 95%]
tests\test_benchmark.py::test_benchmark_fib PASSED                       [100%]

============================== 20 passed in 0.84s ==============================

C:\Users\markian\Desktop\labs\FunctionalProgramming\lab04> python -m mypy .
Success: no issues found in 6 source files

C:\Users\markian\Desktop\labs\FunctionalProgramming\lab04> python -m ruff check .
All checks passed!

C:\Users\markian\Desktop\labs\FunctionalProgramming\lab04> python -m black --check .
All done! 6 files would be left unchanged.
```

Висновок

Під час виконання лабораторної роботи було досліджено структуру рекурсивних обчислень на прикладі знаходження найдовшої спільної підпослідовності (LCS, варіант 12), алгоритму Quicksort, обходів бінарних дерев та послідовності Фібоначчі. З'ясовано механізм навантаження стека викликів у середовищі CPython та причини свідомої відсутності TCO для гарантії прозорого stack trace. Застосування `functools.lru_cache` для мемоізації станів усунуло експоненційну складність наївних рекурсивних підходів, скоротивши час обчислення на кілька порядків. Для запобігання переповненню стека (`RecursionError`) на великих масивах вхідних даних реалізовано ефективні ітеративні альтернативи (табуляція для LCS з пам'яттю *O(min(n, m))*, обходи дерева за допомогою черги/стека та ітеративний Фібоначчі). Коректність рішень та чистоту коду підтверджено повним покриттям pytest-тестами та перевірками лінтерів mypy, ruff і black.